Lecture L20

*Blind review — please score this lecture on row **L20** in the Excel form (1–10 per dimension).*

Learning Objectives
By the end of this lecture, you will be able to:
1. **Recall** the purpose of backpropagation and identify its role in training neural networks.
2. **Understand** how the chain rule is applied in reverse mode to compute gradients efficiently.
3. **Apply** the backpropagation algorithm to compute gradients in a simple multilayer perceptron (MLP) with one hidden layer.
4. **Analyze** common implementation pitfalls such as vanishing gradients and memory constraints.
5. **Evaluate** trade-offs between storing versus recomputing activations during training.

Prerequisites

Before proceeding, ensure familiarity with the following concepts:

- **Partial derivatives and the chain rule**: Given a scalar J that depends on a parameter w_1 through intermediate variables, you can compute $\frac{\partial J}{\partial w_1}$ by multiplying partial derivatives along the computational path.
- **Matrix calculus basics**: You understand how to compute gradients of a scalar loss J with respect to vectors and matrices, such as $\nabla_{\mathbf{W}} J$.
- **Neural network forward pass**: You know how inputs propagate through layers via affine transformations $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ followed by activation functions.
- **Computational graphs**: You can represent a sequence of operations as a directed graph where nodes are variables and edges are functions.

If any of these topics are unfamiliar, pause to review them. The material that follows builds directly on these foundations.

> **Quick Check (Prerequisites)**
> Let $z = \mathbf{w}^\top \mathbf{x} + b$, $a = \sigma(z)$, and $J = (y - a)^2$. Compute $\frac{\partial J}{\partial w_1}$ using the chain rule.
> *(Answer at end of section)*

A Paradox: How Can We Tune Millions of Knobs Blindfolded?

Imagine a sound engineer adjusting 10 million knobs on a mixing board—each controlling a tiny aspect of the audio—without hearing the output until the very end. Once they finally hear the distorted result, they must decide: *Which knob, when turned slightly, would improve the sound most?* And they only get one chance to listen.

Now imagine doing this not once, but thousands of times—each time with a new song, a new desired mix, and a new configuration of knobs. Yet, you're never allowed to adjust while listening. You must predict, in silence, how every knob affects the final output.

This is the training problem in deep learning. Each weight in a neural network is a knob. The loss is the final sound. We can't tweak each one individually—there are too many. Yet, we need to know how *each* affects the final outcome.

Enter **backpropagation**—a method that, after one forward pass (the "listening"), computes *all* the necessary adjustments in a single backward sweep. It's as if the engineer could reconstruct the sensitivity of the final mix to every knob, just from that one playback.

But here's the real puzzle:
If you tried to measure each knob's effect by brute force—nudging one at a time—you'd need 10 million forward passes. Backpropagation does it in *two*: one forward, one backward. How?

Because the network's computation is a chain of known, differentiable operations. And calculus—specifically, reverse-mode application of the chain rule—lets us trace influence backward through that chain with minimal overhead.

> **Try This Yourself**
> Suppose a function is defined as $J = (w_3 \cdot \sigma(w_2 \cdot \sigma(w_1 \cdot x)))^2$. How many forward evaluations would you need to estimate $\nabla_w J$ numerically? How many for backpropagation?
> *Pause and write down your reasoning before reading on.*

Formal Setup: Forward Pass in a Two-Layer MLP

To make ideas concrete, consider a two-layer multilayer perceptron (MLP) with one hidden layer. We fix notation and assumptions for clarity.

Let the input be $\mathbf{x} \in \mathbb{R}^d$. The forward pass proceeds as follows:

$$\begin{aligned}
\mathbf{z}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} && \text{(hidden pre-activation)} \\
\mathbf{h} &= f^{(1)}(\mathbf{z}^{(1)}) && \text{(hidden activation)} \\
z^{(2)} &= \mathbf{W}^{(2)}\mathbf{h} + b^{(2)} && \text{(output pre-activation)} \\
\hat{y} &= f^{(2)}(z^{(2)}) && \text{(prediction)} \\
J &= \frac{1}{2}(y - \hat{y})^2 && \text{(mean squared error loss)}
\end{aligned}$$

We assume scalar output for simplicity. The learnable parameters are:

$$\theta = \left\{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, b^{(2)}\right\}$$

During the **forward pass**, we compute and store: - $\mathbf{z}^{(1)}$, $\mathbf{h}$, $z^{(2)}$, $\hat{y}$, and J

These stored values are essential for the backward pass. For example, computing $\frac{df^{(1)}}{d\mathbf{z}^{(1)}}$ requires $\mathbf{z}^{(1)}$, especially for nonlinearities like ReLU or sigmoid.

> **Misconception Alert**
> Failing to store pre-activations $\mathbf{z}^{(l)}$ is a common implementation error. Without them, activation derivatives cannot be computed correctly during backpropagation.

> **Quick Check (Forward Pass)**
> Let $\mathbf{x} = [1, -1]^\top$, $\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$, $\mathbf{b}^{(1)} = [0, 0]^\top$, and $f^{(1)} = \text{ReLU}$. Compute $\mathbf{h}$.
> *(Answer:* $\mathbf{z}^{(1)} = [1, -2]^\top$*, so* $\mathbf{h} = [1, 0]^\top$*.)*

Core Idea: Reverse-Mode Automatic Differentiation

Backpropagation is a specific instance of **reverse-mode automatic differentiation** (autodiff), which computes gradients of a scalar output with respect to many inputs efficiently.

Let J be the scalar loss. Our goal is to compute $\frac{\partial J}{\partial \theta}$ for every parameter θ.

Rather than differentiating J with respect to each parameter independently, reverse-mode autodiff:

1. Constructs a **computational graph** where nodes represent variables (e.g., $\mathbf{x}, \mathbf{z}^{(1)}, J$) and edges represent operations (e.g., matrix multiplication, activation).
2. Performs a **backward pass**, starting from J, computing $\frac{\partial J}{\partial u}$ for each node u, moving from output to input.

The key recurrence is:

$$\frac{\partial J}{\partial u_i} = \sum_{j \in \text{succ}(i)} \frac{\partial J}{\partial u_j} \cdot \frac{\partial u_j}{\partial u_i}$$

Here, $\text{succ}(i)$ denotes the set of direct successors (dependents) of node u_i. This equation expresses the total sensitivity of J to u_i as a sum over paths through its immediate children.

This is the **chain rule applied in reverse order**—hence the term "reverse mode."

> **Analogy: Hydraulic Network of Valves and Pipes**
> Imagine the forward pass as water flowing through a network of pipes and valves—each operation transforms the flow. The final reservoir level is J. Now, reverse the flow: inject a small pressure pulse at J and watch how it propagates backward. At each junction, the pulse splits or combines based on how sensitive downstream segments were to upstream ones. The pressure at each valve tells you how much adjusting it would change the final level. This backward wave is the gradient—it's not real flow, but *sensitivity* flow.
>
> **Why is this efficient?**
> Each local derivative $\frac{\partial u_j}{\partial u_i}$ involves only one operation and is cheap to compute. By reusing $\frac{\partial J}{\partial u_j}$ once it is known, we avoid redundant calculations. The full gradient vector is computed in time proportional to a constant multiple of the forward pass.
>
> **Try This Yourself**
> Draw the computational graph for $J = (w_2 \cdot \sigma(w_1 \cdot x))^2$. Mark the forward direction with green arrows, the backward gradient flow with red. At which node does the gradient "split"? Where does it "merge"?
> *Now simulate the backward pass: assign a gradient of 1 to J, then propagate it step by step. What is $\frac{\partial J}{\partial w_1}$?*
>
> **Quick Check (Reverse Mode)**
> In reverse-mode differentiation, do we compute gradients from input to output, or output to input?
> *(Answer: Output → input — backwards through the computational graph.)*

Worked Example: Backpropagation in a Two-Layer MLP

We now walk through a complete numerical example of backpropagation in a two-layer MLP.

Forward Pass (Numerical Example) Given: - $\mathbf{x} = [2, 3]^\top$ - $\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$, $\mathbf{b}^{(1)} = [0, 1]^\top$ - $f^{(1)} = \text{ReLU}$ - $\mathbf{W}^{(2)} = [1, -2]$, $b^{(2)} = 0$ - $f^{(2)} = \text{identity}$ (linear output) - True label: $y = 1$

Compute forward:

1. $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
2. $\mathbf{h} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
3. $z^{(2)} = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)} = [1, -2] \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0 = 2 - 4 = -2$
4. $\hat{y} = f^{(2)}(z^{(2)}) = -2$
5. $J = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - (-2))^2 = \frac{1}{2}(9) = 4.5$

We store $\mathbf{z}^{(1)}, \mathbf{h}, z^{(2)}$ for use in the backward pass.

> **Try This Yourself**
> Before proceeding: predict which weight in $\mathbf{W}^{(1)}$ has the largest impact on J. Is it $W_{11}^{(1)}$ or $W_{22}^{(1)}$? Write down your reasoning based on signal path and sign.

Backward Pass We now compute gradients via backpropagation, starting from J.

1. $\frac{\partial J}{\partial \hat{y}} = -(y - \hat{y}) = -(1 - (-2)) = -3$
2. $\frac{\partial J}{\partial z^{(2)}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{(2)}} = -3 \cdot 1 = -3$
3. $\nabla_{\mathbf{h}} J = \left(\frac{\partial z^{(2)}}{\partial \mathbf{h}}\right)^\top \frac{\partial J}{\partial z^{(2)}} = (\mathbf{W}^{(2)})^\top \cdot (-3) = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \cdot (-3) = \begin{bmatrix} -3 \\ 6 \end{bmatrix}$
4. $\nabla_{\mathbf{z}^{(1)}} J = \nabla_{\mathbf{h}} J \odot f^{(1)\prime}(\mathbf{z}^{(1)}) = \begin{bmatrix} -3 \\ 6 \end{bmatrix} \odot \mathbf{1} = \begin{bmatrix} -3 \\ 6 \end{bmatrix}$
   (since $\mathbf{z}^{(1)} = [2, 2]^\top > 0$, ReLU derivative is 1)
5. $\nabla_{\mathbf{W}^{(1)}} J = \nabla_{\mathbf{z}^{(1)}} J \cdot \mathbf{x}^\top = \begin{bmatrix} -3 \\ 6 \end{bmatrix} \begin{bmatrix} 2 & 3 \end{bmatrix} = \begin{bmatrix} -6 & -9 \\ 12 & 18 \end{bmatrix}$
6. $\nabla_{\mathbf{b}^{(1)}} J = \nabla_{\mathbf{z}^{(1)}} J = \begin{bmatrix} -3 \\ 6 \end{bmatrix}$
7. $\nabla_{\mathbf{W}^{(2)}} J = \frac{\partial J}{\partial z^{(2)}} \cdot \mathbf{h}^\top = (-3) \cdot \begin{bmatrix} 2 & 2 \end{bmatrix} = \begin{bmatrix} -6 & -6 \end{bmatrix}$
8. $\nabla_{b^{(2)}} J = \frac{\partial J}{\partial z^{(2)}} = -3$

> **Insight from the Numbers**
> The gradient $\nabla_{\mathbf{W}^{(1)}} J$ shows that increasing $W_{21}^{(1)}$ or $W_{22}^{(1)}$ will reduce J the most—because their gradients are positive (so increasing them moves weights in the negative gradient direction). This aligns with our earlier intuition: the second hidden unit contributes more strongly to error reduction due to larger gradient magnitude.

> **Try This Yourself**
> Suppose we used sigmoid instead of ReLU for $f^{(1)}$. Recompute $\nabla_{\mathbf{z}^{(1)}} J$ using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. What happens to the gradient magnitude? What if $z^{(1)}$ had been very large or very negative?

Implementation Challenges and Trade-offs

Backpropagation is mathematically elegant—but real implementations face practical constraints.

Vanishing Gradients When many nonlinearities (e.g., sigmoid, tanh) are stacked, gradients can shrink exponentially as they propagate backward. Early layers receive tiny updates, slowing or halting learning.

> **Analogy: Whispering Down the Lane**
> Imagine a message passed through 10 people, each whispering it fainter than the last. By the end, the final recipient hears almost nothing. In deep networks, gradients are the message—activation functions like sigmoid act as volume knobs that dampen the signal layer by layer.

This is why ReLU and its variants (Leaky ReLU, ELU) dominate modern architectures—they preserve gradient magnitude for positive inputs.

Memory vs. Recomputation Backpropagation requires storing all pre-activations $\mathbf{z}^{(l)}$. For deep networks, this can exhaust memory.

An alternative: **gradient checkpointing**. Store only some activations and recompute the rest during backward pass. It trades computation for memory—like saving only key frames in a video and redrawing the rest when needed.

> **Try This Yourself**
> In a 10-layer network, storing all $\mathbf{z}^{(l)}$ uses 10 units of memory. If you store only layers 1, 5, and 10, how much memory do you save? What is the recomputation cost during backprop? Sketch the forward and backward phases.

Summary

Backpropagation is not magic—it's calculus, organized.
It exploits the chain rule in reverse to compute sensitivities efficiently, turning a million-blind-knob problem into a two-pass routine.

But efficiency comes with trade-offs: memory demands, numerical stability, and architectural constraints like vanishing gradients.

Understanding these is not just academic—it's essential for debugging models that won't learn, training networks on limited hardware, and designing architectures that *can* be trained.

> **Final Challenge**
> Can you design a 3-layer network where backpropagation *must* fail due to gradient vanishing—even with perfect initialization? What activation function would cause this? What would the gradient norms look like across layers?

> **Prerequisite Answer**
> $\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1} = -2(y-a) \cdot \sigma'(z) \cdot x_1$